

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
Топологическая сортировка по схеме Кана, Тарьяна и Демукрона.
Красно-черное дерево

Выполнил:
Студент гр. N3248
Жук Анастасия Валерьевна

Подпись: _____

Проверил:
Ерофеев Сергей Анатольевич

Подпись: _____

Санкт-Петербург
2025 г.

Техническое задание

Разработать программу топологической сортировки по схеме Кана, Тарьяна или Демукрона по выбору пользователя. Числа из файла записываются в очередь на базе динамического массива. Записать результат в красно-черное дерево.

Переменные

Константы (#define)

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
INITIAL_BUF_SIZE	Начальный размер буфера для чтения строк	int	100	4

Динамическая очередь (DynamicQueue)

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
data	Динамический массив элементов	int*	от -2 147 483 648 до 2 147 483 647 (1 int)	4 * размер массива
capacity	Вместимость очереди	int	от 0 до 2 147 483 647	4
size	Текущий размер очереди	int	от 0 до 2 147 483 647	4
first	Индекс первого элемента	int	от 0 до 2 147 483 647	4
last	Индекс последнего элемента	int	от 0 до 2 147 483 647	4

Красно-черное дерево (RBNode и RBTree)

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
value	Значение узла	int	от -2 147 483 648 до 2 147 483 647	4
color	Цвет узла (RED/BLACK)	Color	0 или 1	4

left	Левый потомок	RBNode*	Адрес памяти	8
right	Правый потомок	RBNode*	Адрес памяти	8
parent	Родительский узел	RBNode*	Адрес памяти	8
root	Корень дерева	RBNode*	Адрес памяти	8
nil	Специальный листовой узел	RBNode*	Адрес памяти	8

Функции работы с очередью

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
q	Указатель на очередь	DynamicQueue*	Адрес памяти	8
value	Добавляемое/ извлекаемое значение	int	от -2 147 483 648 до 2 147 483 647	4
new_data	Новый массив данных	int*	от -2 147 483 648 до 2 147 483 647 (1 int)	4 * размер массива
new_capacity	Новая вместимость	int	от 0 до 2 147 483 647	4

Функции работы с матрицей

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
matrix	Указатель на матрицу	DynamicQueue*	Адрес памяти	8
row	Индекс строки	int	от 0 до 2 147 483 647	4
col	Индекс столбца	int	от 0 до 2 147 483 647	4
n	Размер матрицы	int	от 0 до 2 147 483 647	4
pos	Линейная позиция	int	от 0 до 2 147 483 647	4
index	Реальный индекс в буфере	int	от 0 до 2 147 483 647	4

Функции работы с красно-черным деревом

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
tree	Указатель на дерево	RBTree*	Адрес памяти	8
x, y, z	Временные узлы	RBNode*	Адрес памяти	8
value	Вставляемое значение	int	от -2 147 483 648 до 2 147 483 647	4
node	Текущий узел (для обхода)	RBNode*	Адрес памяти	8
nil	Специальный лист	RBNode*	Адрес памяти	8

Общие для всех алгоритмов сортировки

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
matrix	Матрица смежности	DynamicQueue*	Адрес памяти	8
n	Количество вершин	int	от 0 до 2 147 483 647	4
result_size	Размер результата	int*	от 0 до 2 147 483 647	4
result	Массив результата	int*	от -2 147 483 648 до 2 147 483 647 (1 int)	n * 4

Алгоритм Кана

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
in_degree	Массив полустепеней захода	int*	от 0 до 2 147 483 647 (1 int)	n * 4
q	Очередь для вершин степени 0	DynamicQueue*	Адрес памяти	8
v, i, j	Итераторы	int	от 0 до 2 147 483 647	4

Алгоритм Тарьяна

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
colors	Массив цветов вершин	int*	0 или 1 (1 int)	n * 4
q	Очередь для хранения результата	DynamicQueue*	Адрес памяти	8
v, u	Итераторы	int	от 0 до 2 147 483 647	4

Алгоритм Демукрона

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
in_degree	Массив полустепеней захода	int*	от 0 до 2 147 483 647 (1 int)	n * 4
q	Очередь для текущего уровня	DynamicQueue*	Адрес памяти	8
v, i, j	Итераторы	int	от 0 до 2 147 483 647	4

Основная функция

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
algorithm_choice	Выбор алгоритма	int	1, 2 или 3	4
filename	Имя файла	const char*	"matrix.txt"	Длина строки (10)
file	Указатель на файл	FILE*	Адрес памяти	8
rows, cols	Размеры матрицы	int	от 0 до 2 147 483 647	4
line	Считанная строка	char*	Динамическая память	Зависит от длины строки

current_cols	Текущее количество столбцов	int	от 0 до 2 147 483 647	4
ptr, token	Указатели для разбора строки	char*	Адрес памяти	8
value	Значение элемента	double	0.0 или 1.0	8
matrix	Матрица смежности	DynamicQueue*	Адрес памяти	8
sorted_vertices	Отсортированные вершины	int*	от 0 до 2 147 483 647 (1 int)	n * 4
tree	Красно-черное дерево	RBTree*	Адрес памяти	8
i, j	Итераторы	int	от 0 до 2 147 483 647	4

Вспомогательные функции

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
file	Указатель на файл	FILE*	Адрес памяти	8
buf_size	Размер буфера	size_t	от 0 до 2 ⁶⁴	8
buffer	Буфер для строки	char*	Динамическая память	Зависит от строки
pos	Текущая позиция	size_t	от 0 до 2 ⁶⁴	8
c	Считанный символ	int	от 0 до 2 147 483 647	4
new_buf	Новый буфер	char*	Динамическая память	Зависит от строки
str	Проверяемая строка	const char*	Динамическая память	Зависит от строки
endptr	Указатель на конец числа	char*	Динамическая память	Зависит от строки

Функциональные маршруты

Перед тем как приступить к самой сортировке, необходимо считать из файла матрицу смежности или весов графа. Для топологической сортировки достаточно только матрицы смежности (матрицы, у которой на пересечении i -ой строки и j -ого столбца стоит 1, если существует дуга из вершины i в вершину j , и 0 в противном случае), но также можно рассмотреть матрицу весов, заменяя вес дуги из вершины i в вершину j на 1. Веса никак не влияют на сортировку, поэтому можем от них избавиться и перейти к матрице смежности.

Итак, для того, чтобы перейти к сортировке, нужно проверить следующие свойства матрицы:

1. Матрица состоит из численных значений, не содержит букв и слов;
2. Матрица не пустая;
3. Матрица квадратная, то есть количество столбцов в ней равно количеству строк.

Если в строке присутствуют пустые строки и/или лишние пробелы, то в процессе считывания матрицы из файла в массив необходимо исключить лишние символы.

На первом этапе нужно проверить содержимое файла, чтобы определить, подходит ли оно нам: содержится ли там матрица и удовлетворяет ли она всем вышеописанным условиям.

Если содержимое файла нам подходит, то мы можем записывать значения матрицы в дек, предварительно заменив веса на 1. Матрица будет храниться в виде очереди на базе динамического массива, поэтому для дальнейшей работы с ней нужно запомнить ее размерность.

Когда матрица была успешно считана и записана в очередь, можно переходить к самой топологической сортировке. У пользователя есть возможность выбрать одну из трех сортировок: Кана, Тарьяна или Демукрона.

Топологическая сортировка — это процесс упорядочивания вершин ациклического ориентированного графа согласно частичному порядку, заданному рёбрами орграфа на множестве его вершин.

Алгоритм Кана — один из методов топологической сортировки, основанный на постепенном удалении вершин с нулевой полустепенью захода.

Алгоритм топологической сортировки по схеме Кана:

1. Вычисляем полустепень захода (количество входящих рёбер) для каждой вершины.
2. Создаём очередь (или дек) для вершин с нулевой полустепенью захода.
3. Извлекаем вершину v из очереди и добавляем её в результат.
4. Уменьшаем полустепени захода для всех соседей v (т.е. вершин, достижимых из v).
 - Если полустепень захода соседа стала нулевой, добавляем его в очередь.
5. Выполняем 3 и 4 пункты до тех пор, пока очередь не станет пустой
6. Если в результате оказались все вершины — граф ациклический, и топологическая сортировка выполнена.
7. Если остались вершины — граф содержит цикл, и сортировка невозможна.

Алгоритм Тарьяна – метод топологической сортировки, заключающийся в обходе всех вершин, используя поиск в глубину.

В самом начале все вершины белые. Для каждой из них осуществляется шаг алгоритма:

- если вершина чёрная, то ничего делать не надо;
- если вершина серая, то найден цикл, а значит топологическая сортировка невозможна;
- если вершина белая, то:
 - она красится в серый;
 - применяется шаг алгоритма для всех вершин, в которые можно попасть из текущей;
 - вершина красится в чёрный и помещается в окончательный список.

Полученный список, выведенный в обратном порядке, даст нам отсортированный порядок вершин графа.

Метод Демукрона — это алгоритм топологической сортировки, который использует разбиение графа на уровни.

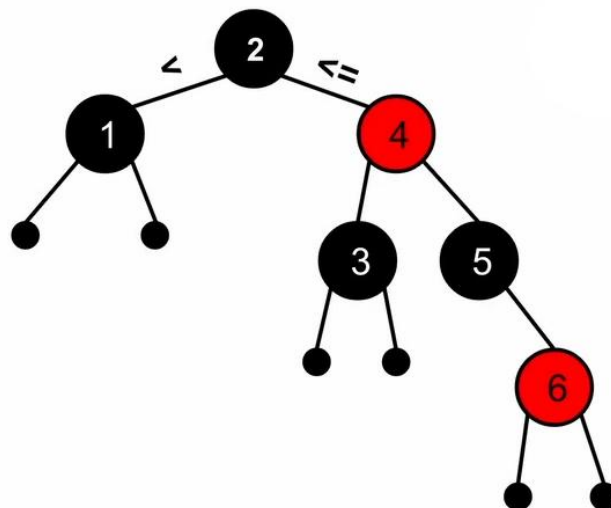
Алгоритм топологической сортировки Демукрона:

1. Вычислить полустепеней захода: для каждой вершины $v \in V$ вычисляем количество входящих ребер.
2. Найти все вершины с нулевой полустепенью захода.
3. Добавить их в текущий уровень и удалить из графа.
4. Обновить полустепени захода для всех вершин, в которые входят рёбра из удалённых вершин.
5. Повторять пункты 2 – 4, пока все вершины не будут распределены по уровням.

Если после обработки остались вершины с ненулевыми полустепенями захода, значит, граф содержит цикл, и топологическая сортировка невозможна.

После последовательного объединения уровней (первый + второй + ...) получится отсортированный список вершин. Если на одном уровне находится несколько вершин, то в списке их можно располагать в произвольном порядке.

Красно-черное дерево

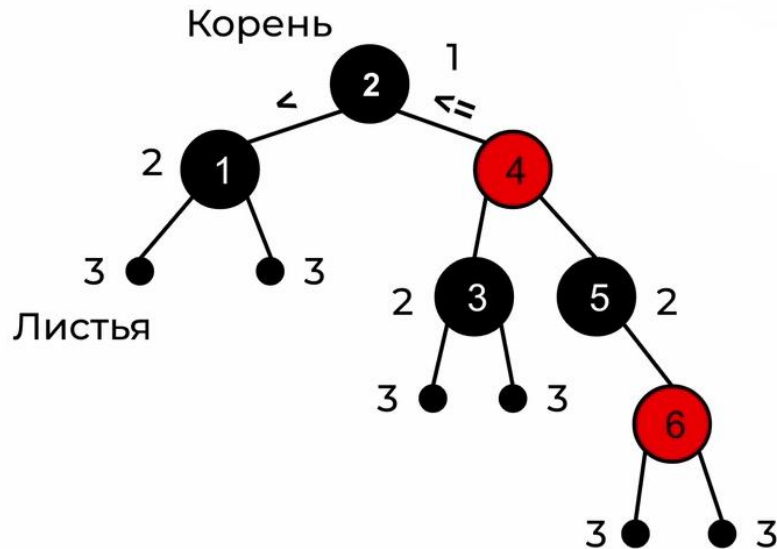


Красно-чёрное дерево – один из видов самобалансирующихся двоичных деревьев поиска, гарантирующих логарифмический рост высоты дерева от числа узлов и позволяющее быстро выполнять основные операции дерева поиска: добавление, удаление и поиск узла. Сбалансированность достигается за счёт введения дополнительного атрибута узла дерева — «цвета». Этот атрибут может принимать одно из двух возможных значений — «чёрный» или «красный».

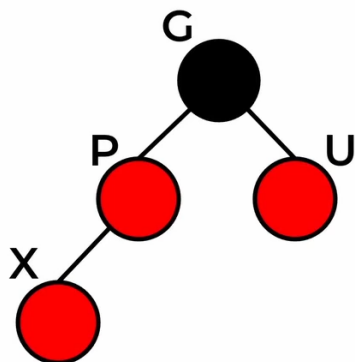
Условия оптимально построенного красно-черного дерева:

1. Цвет узла черный/красный;
2. Корень всегда черный;

3. Листья всегда черные (являются фиктивными и не содержат данных, но относятся к дереву; для экономии памяти их можно сделать одним общим фиктивным листом);
4. Каждый красный узел должен иметь 2 черных узла, черный может иметь черных сыновей;
5. Пути от узла к его листьям должен содержать одинаковое количество черных узлов (это черная высота).



Черная высота = 3



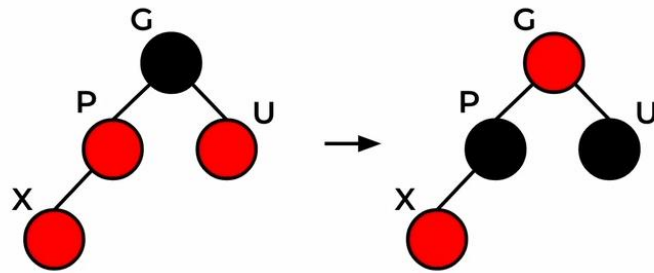
X - сын, добавляемый элемент
P - отец, вышестоящий узел
G - дед, вышестоящий узел отца
U - дядя, соседний узел от отца

Действие (добавление/удаление) + Балансировка = Поддержание скорости алгоритма

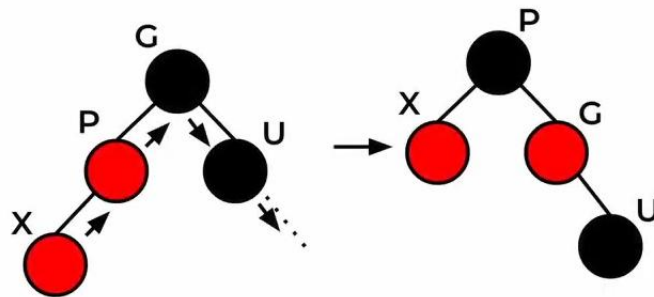
Балансировка — это процесс проверки структуры красно-черного дерева на соответствие пяти ключевым условиям его оптимального построения. Если обнаруживается нарушение какого-либо из правил, выполняется серия преобразований (поворотов и перекрашиваний узлов), приводящих дерево к корректному состоянию. Основная цель балансировки — обеспечить сбалансированность дерева, гарантируя тем самым логарифмическую сложность операций поиска, вставки и удаления.

Случаи, при которых проводится балансировка:

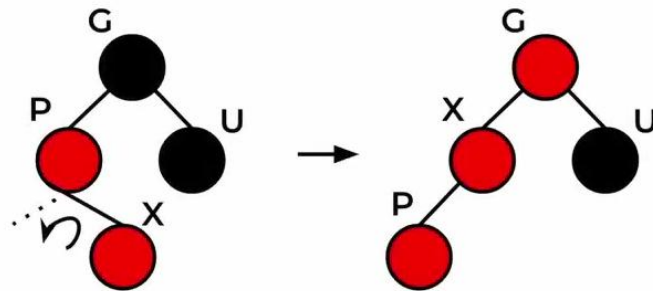
- Красный дядя – перекрашиваем;



- Черный дядя, папа и дед по одной линии – большой поворот;

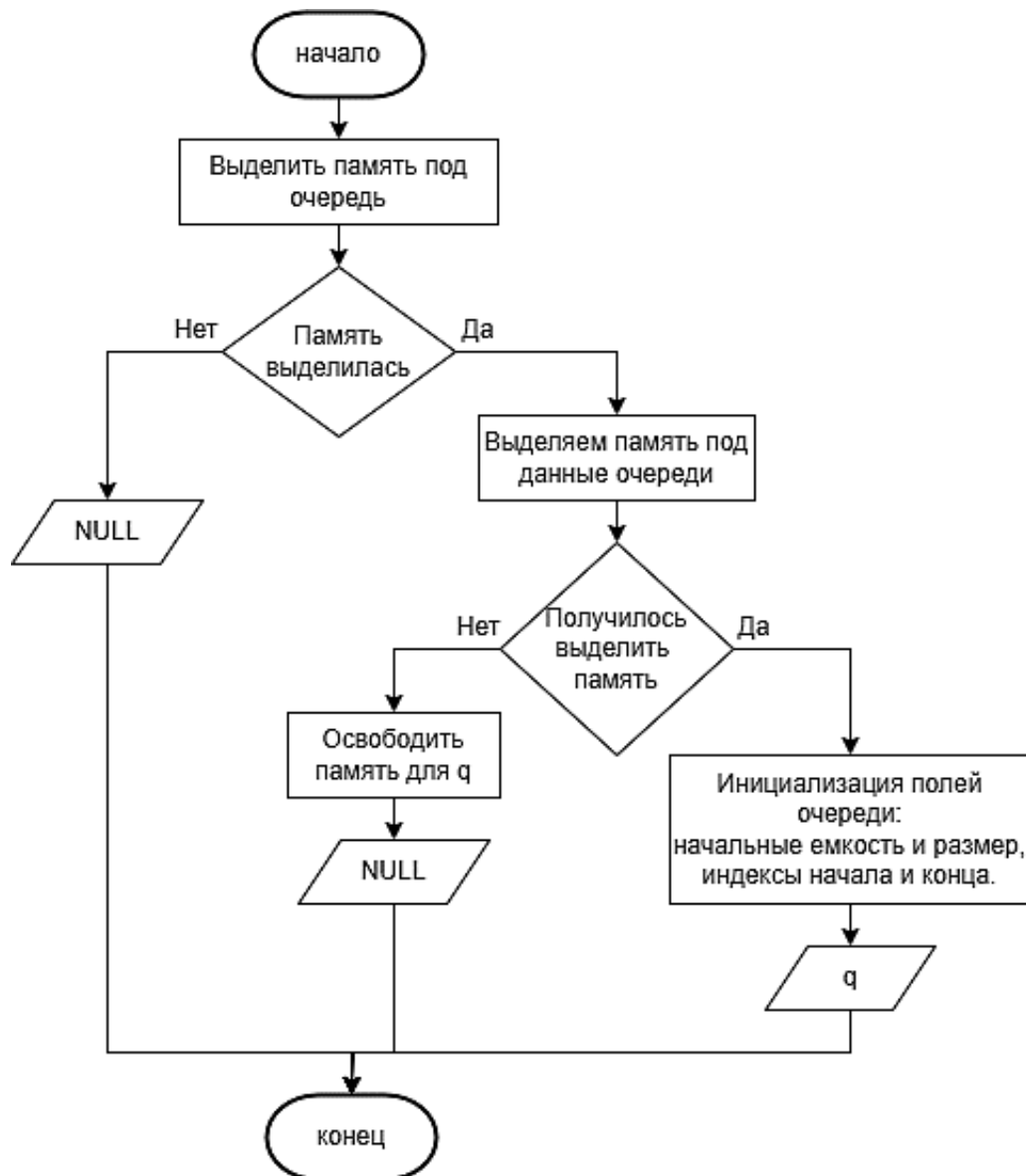


- Черный дядя – поворот + большой поворот.

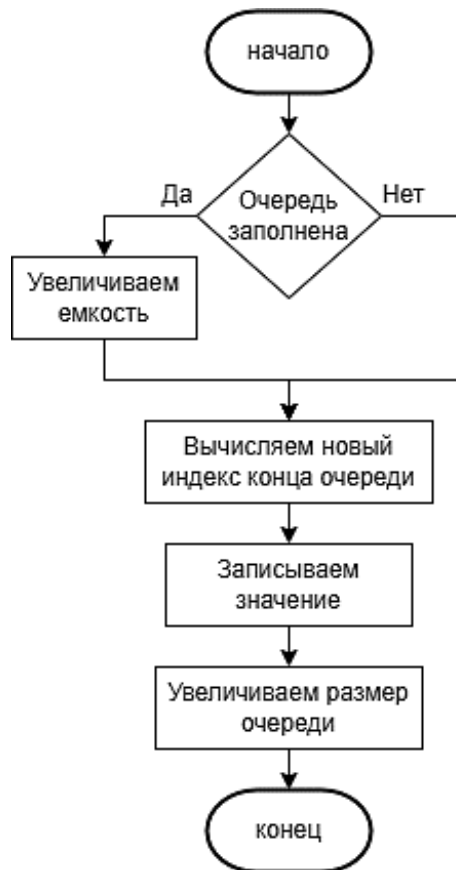


Блок-схемы

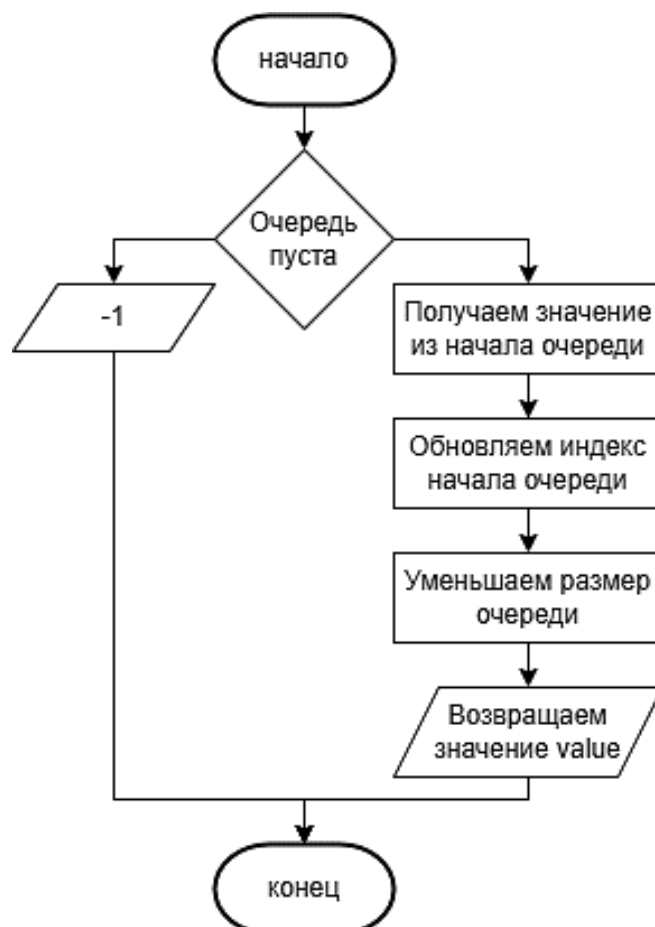
Функция `create_queue(int size)`



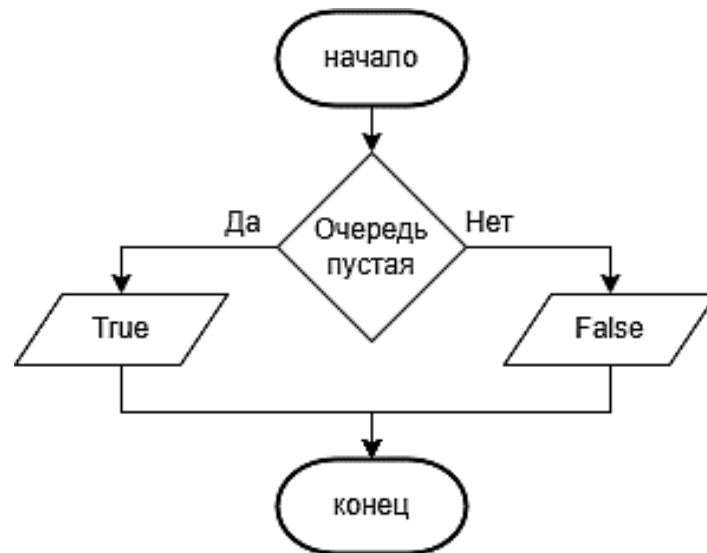
Функция **enqueue(DynamicQueue* q, int value)**



Функция **dequeue(DynamicQueue* q)**



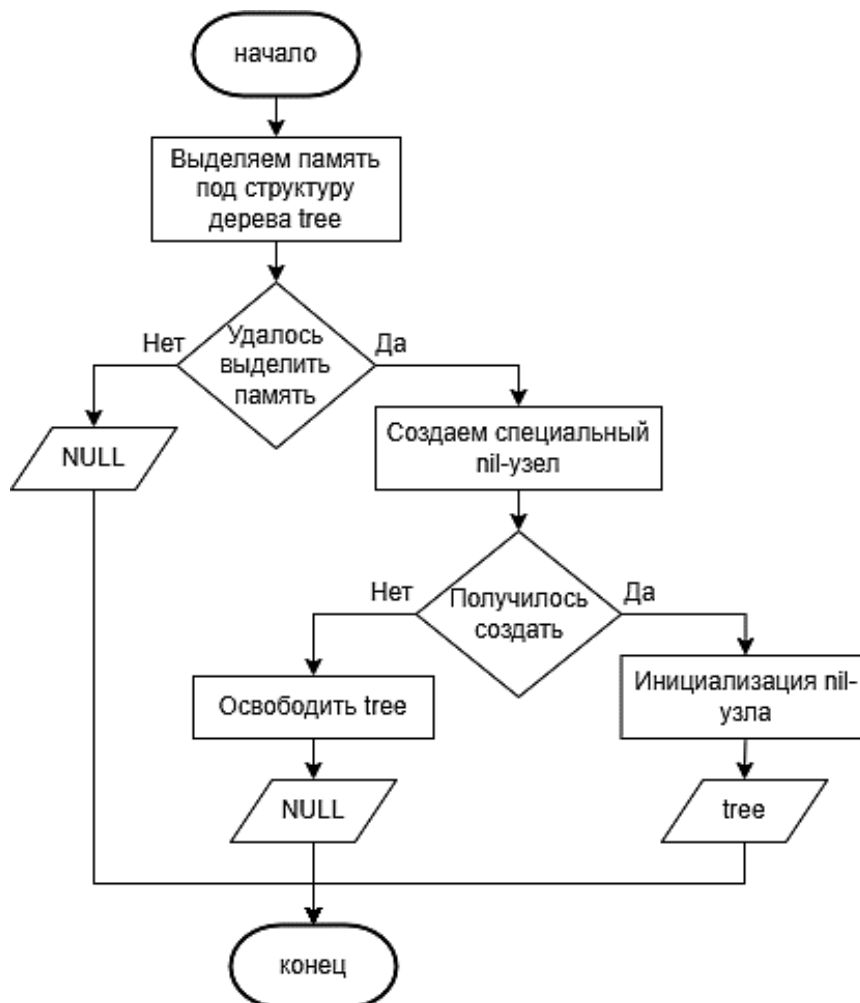
Функции `is_empty(DynamicQueue* q)` и `free_queue(DynamicQueue* q)`



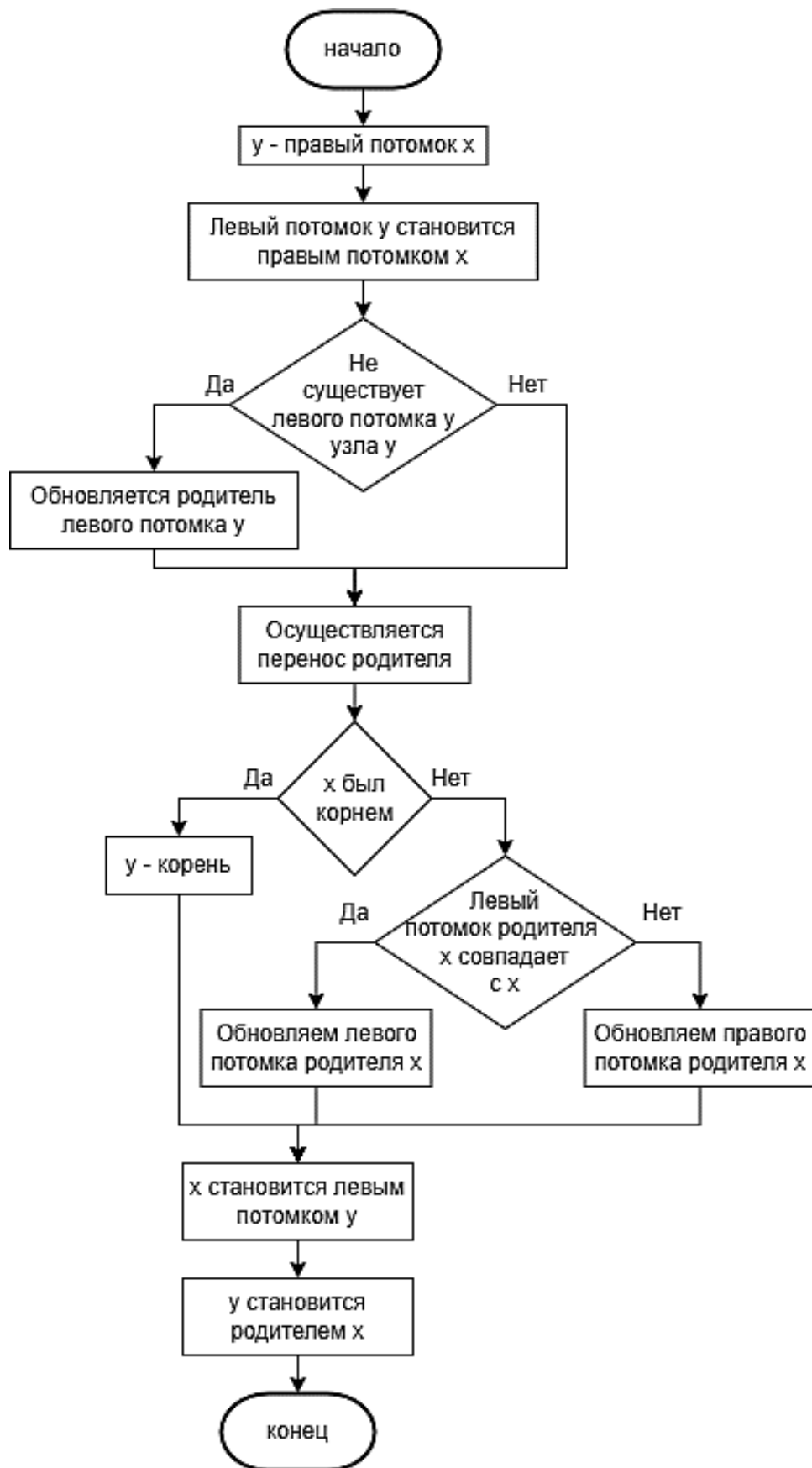
Функция **get_matrix_element(DynamicQueue* matrix, int row, int col, int n)**



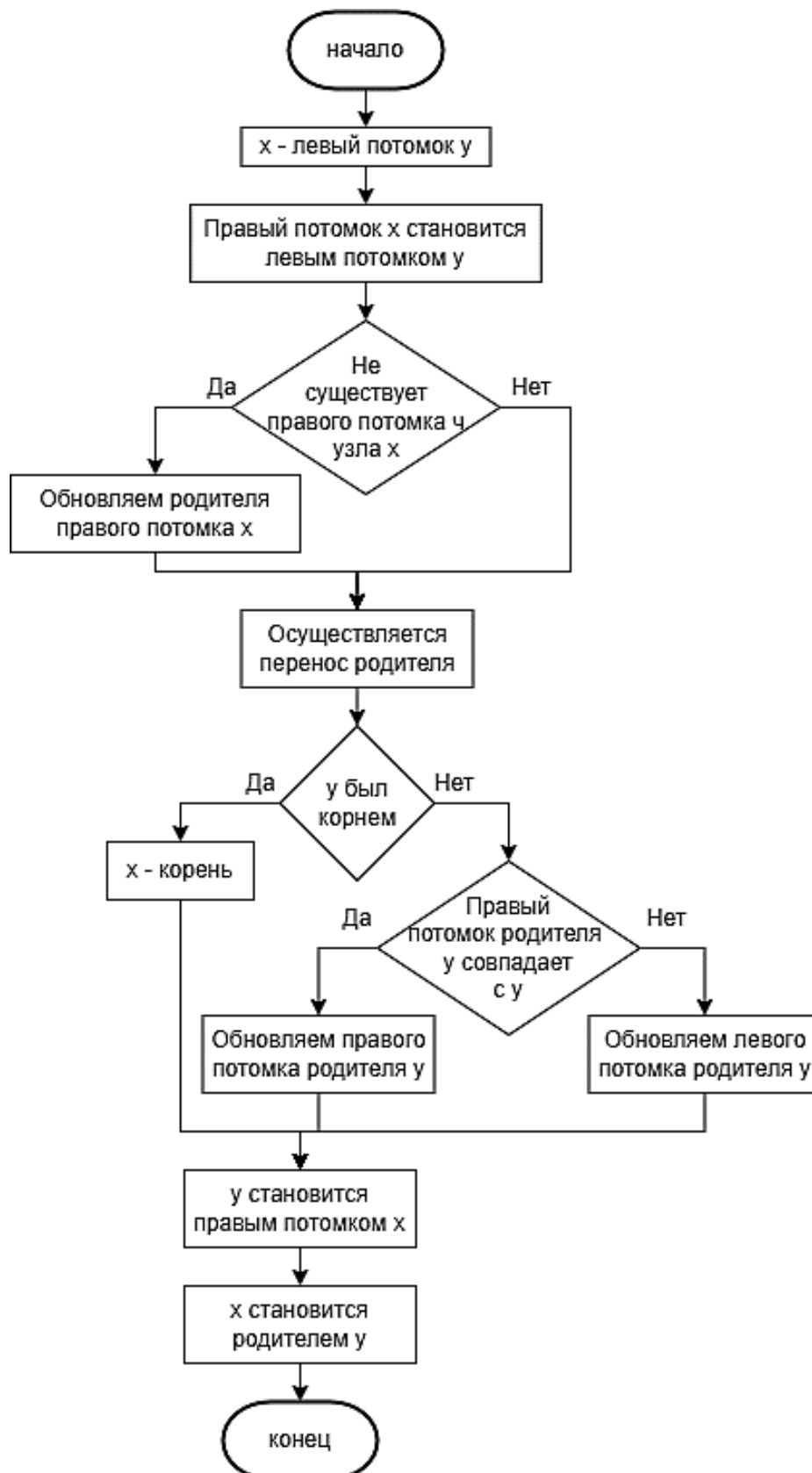
Функция **create_rb_tree()**



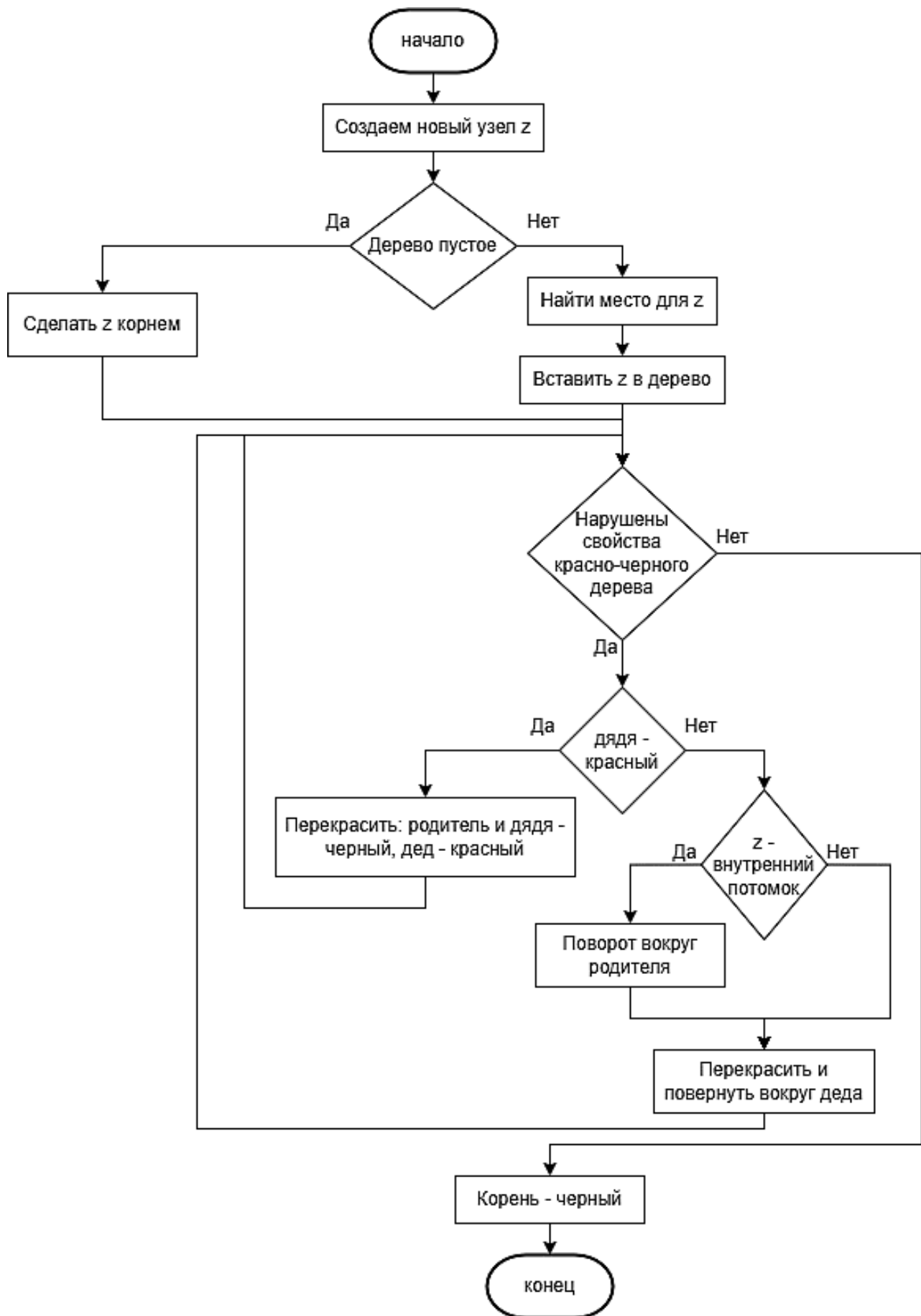
Функция **left_rotate(RBTree* tree, RBNode* x)**



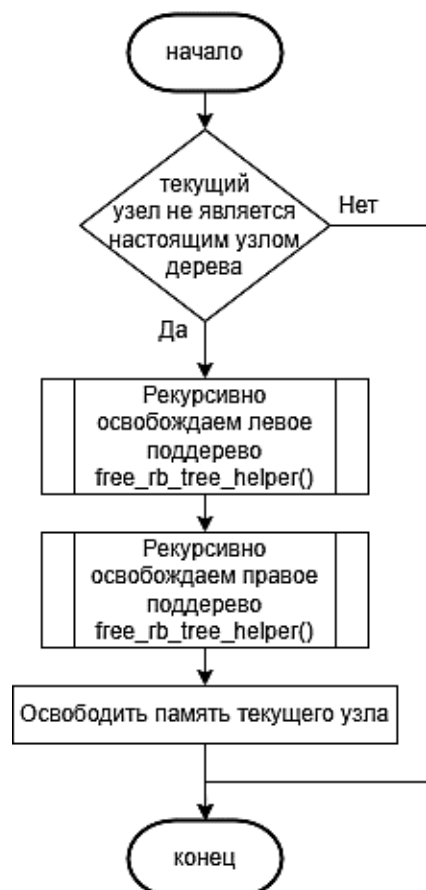
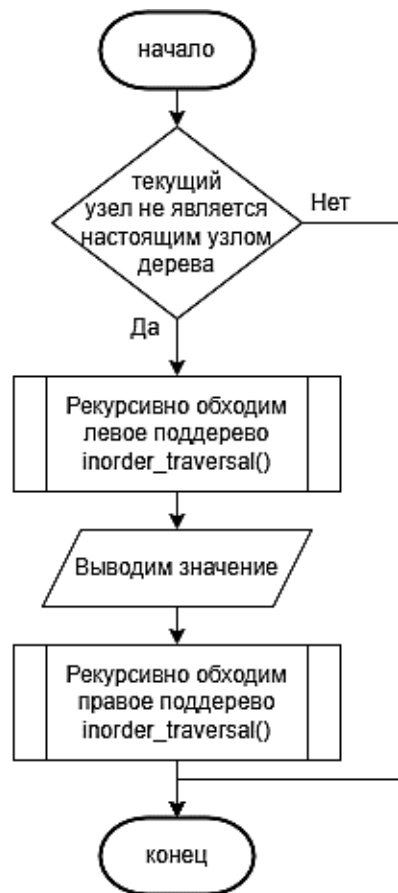
Функция **right_rotate(RBTree* tree, RBNode* y)**



Функция `rb_insert(RBTree* tree, int value)`



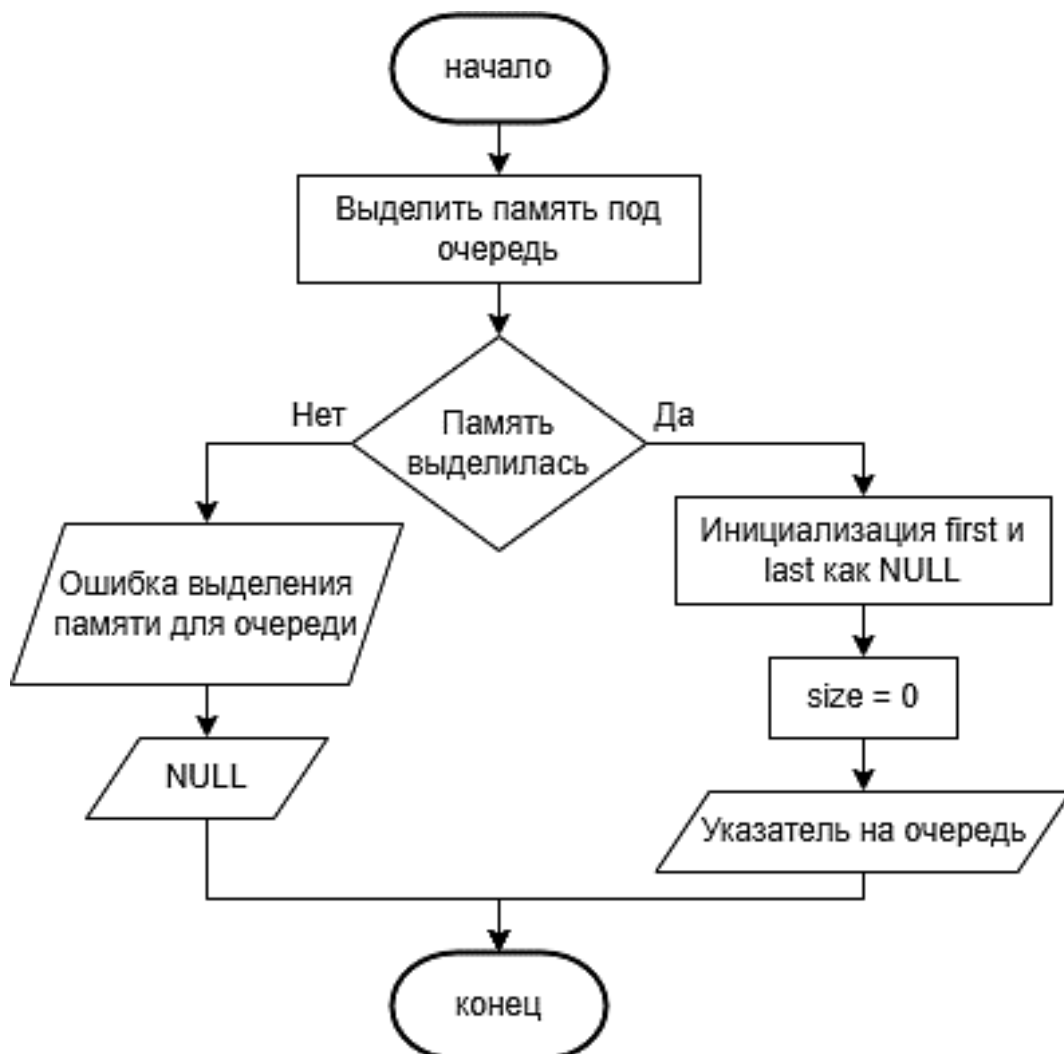
Функции `inorder_traversal(RBNode* node, RBNode* nil)` и `free_rb_tree_helper(RBNode* node, RBNode* nil)`



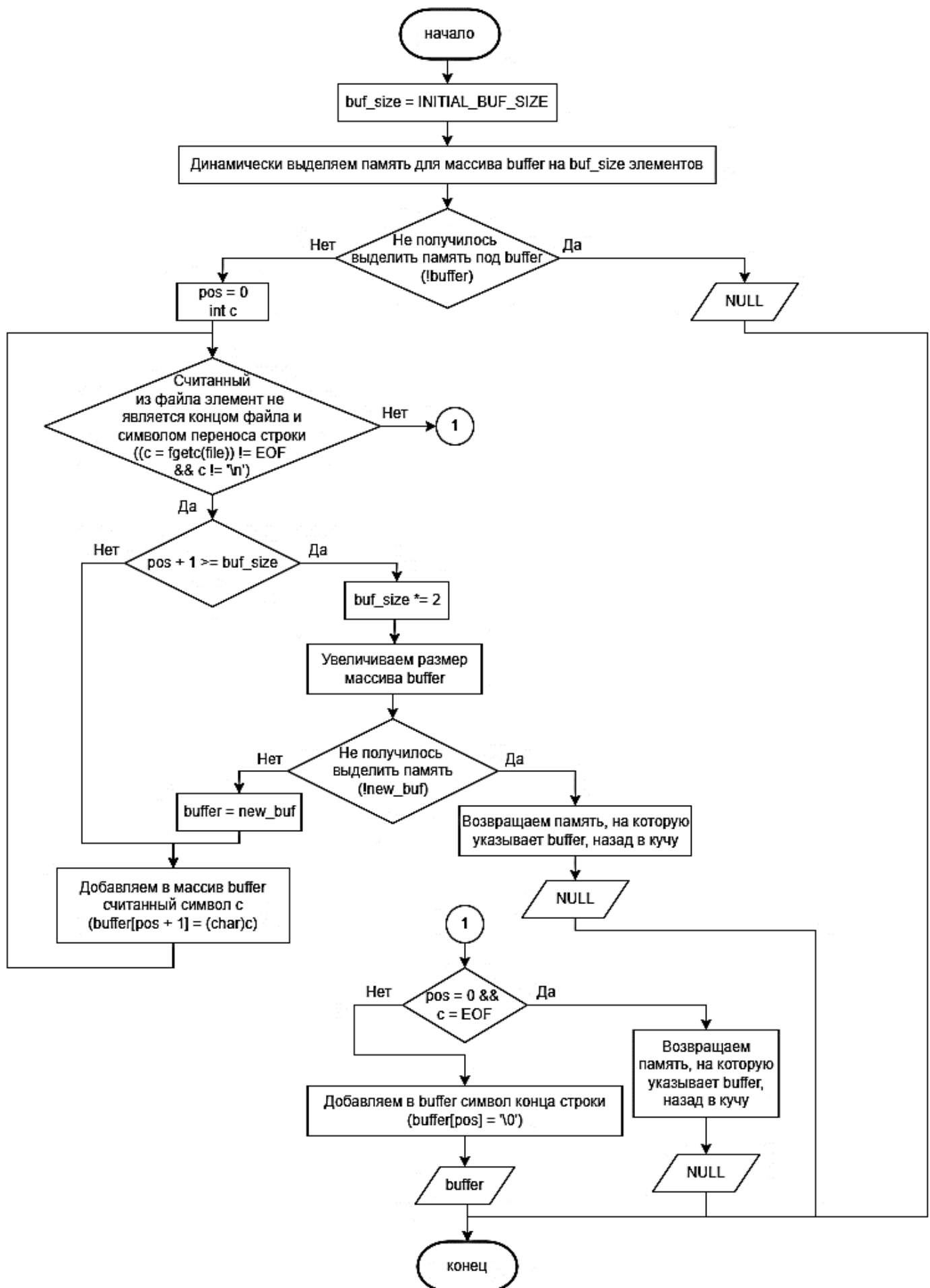
Функция **free_rb_tree(RBTree* tree)**



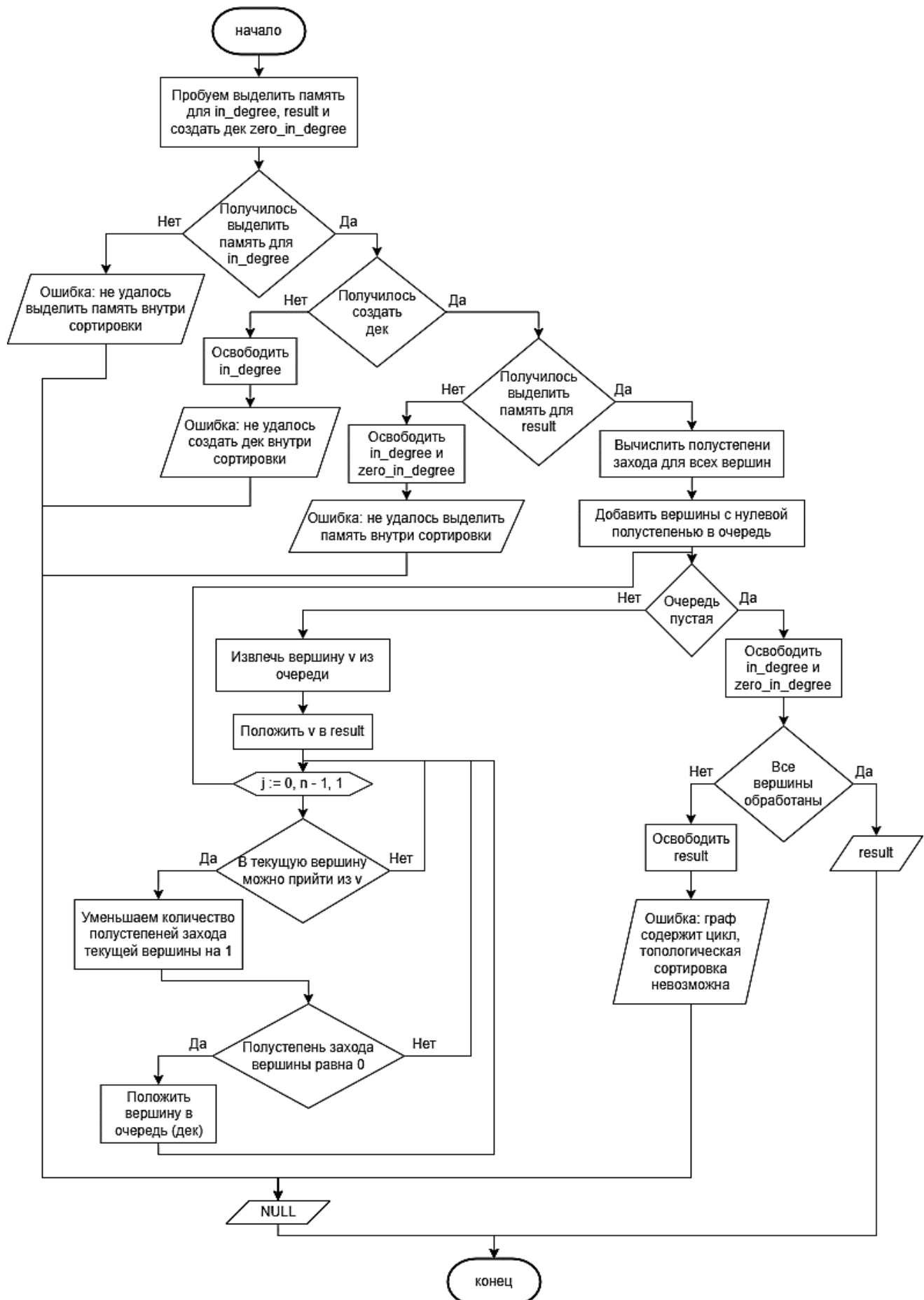
Функция **is_valid_number(const char *str)**



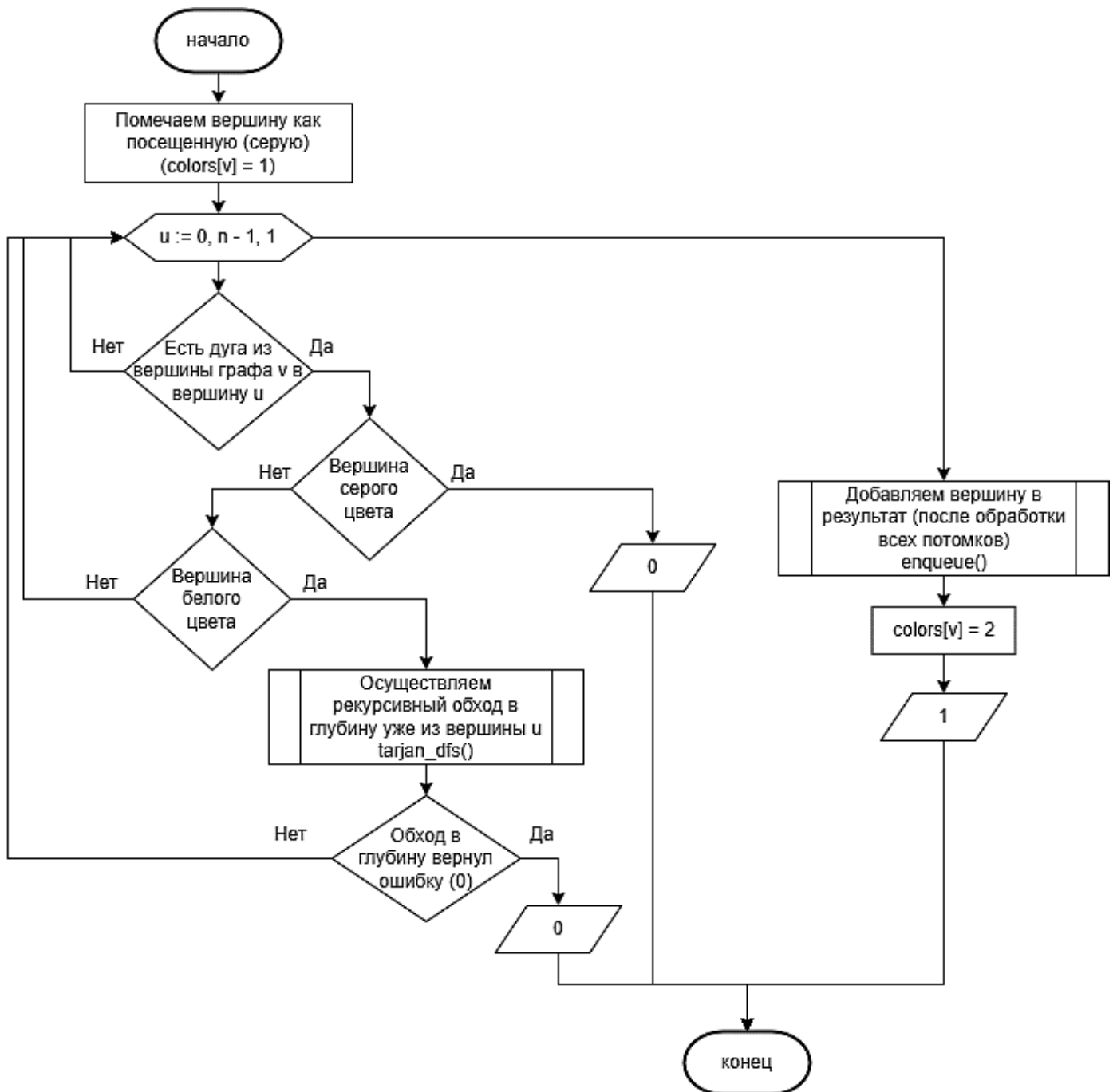
Функция `read_dynamic_line(FILE* file)`



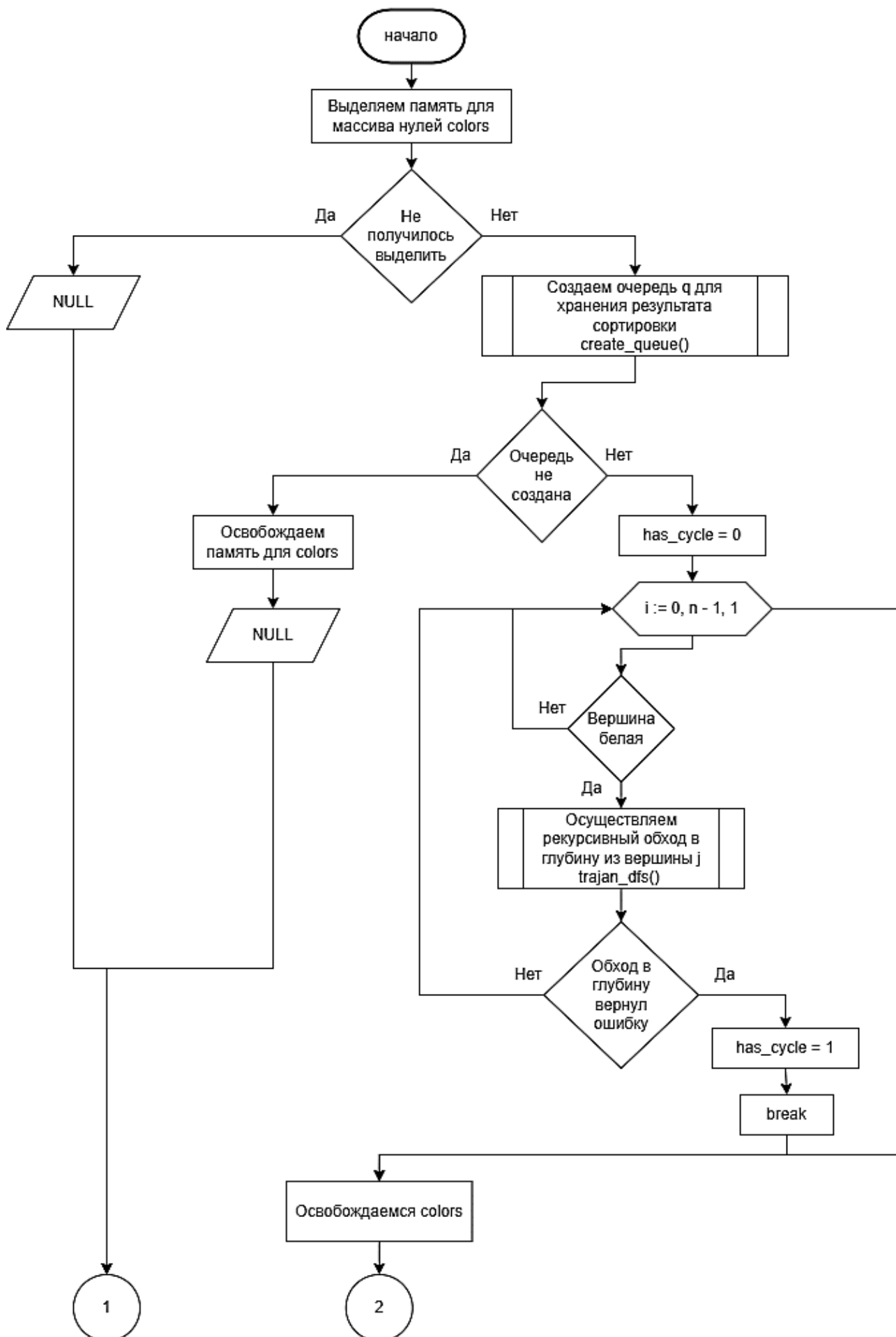
Функция `kahn_topological_sort(DynamicQueue* matrix, int n, int* result_size)`

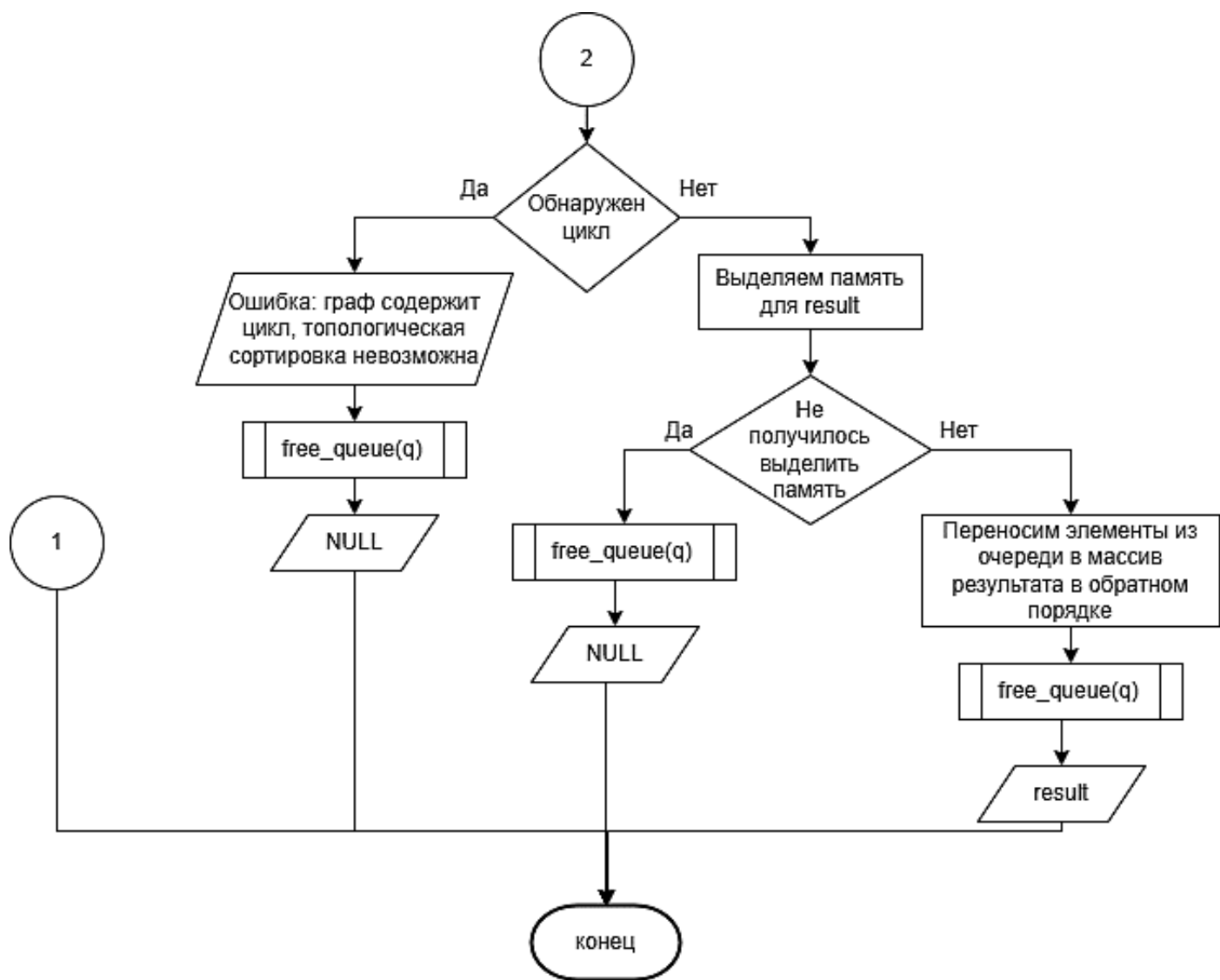


Функция **tarjan_dfs(int v, DynamicQueue* matrix, int n, int* colors, DynamicQueue* result)**

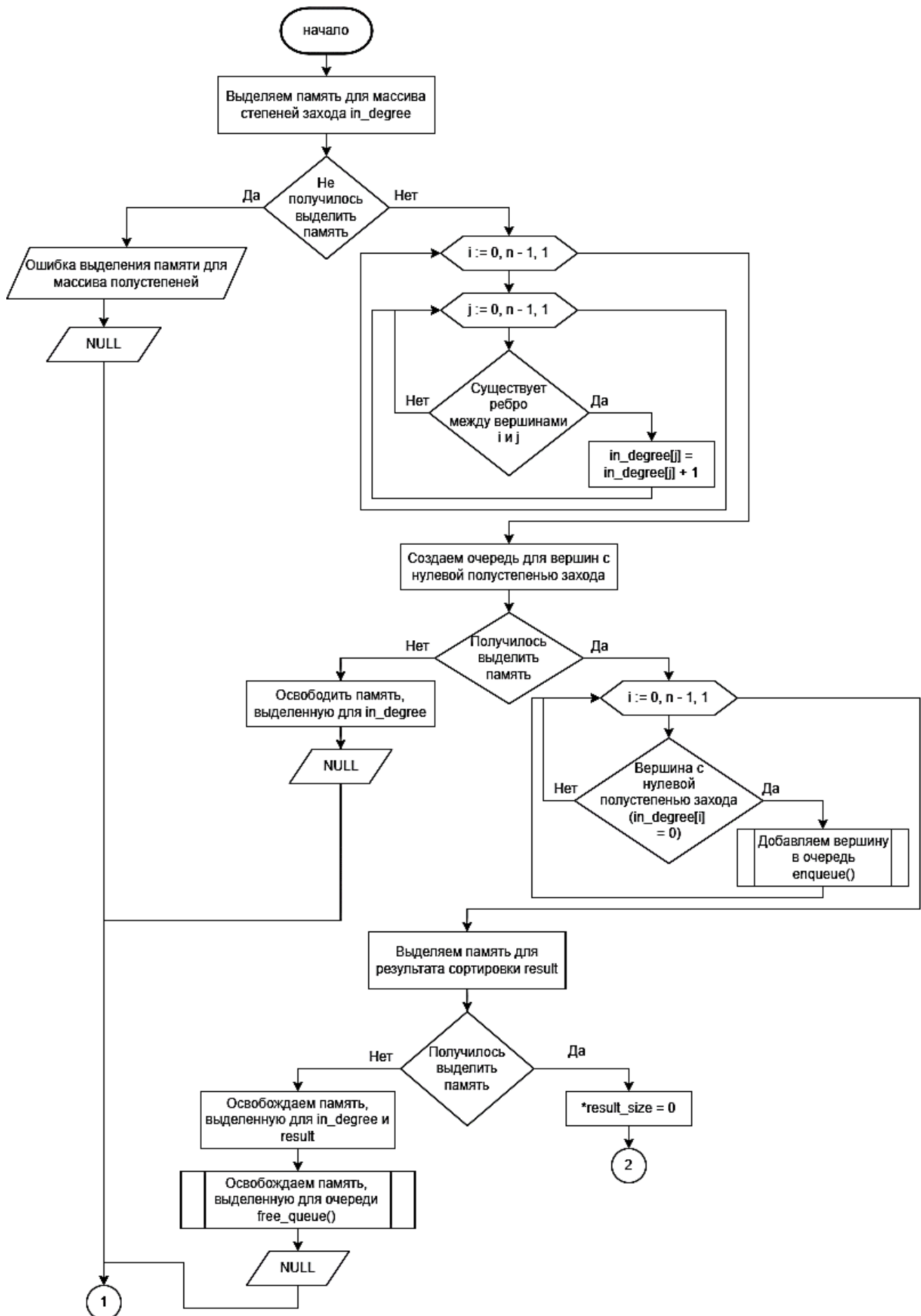


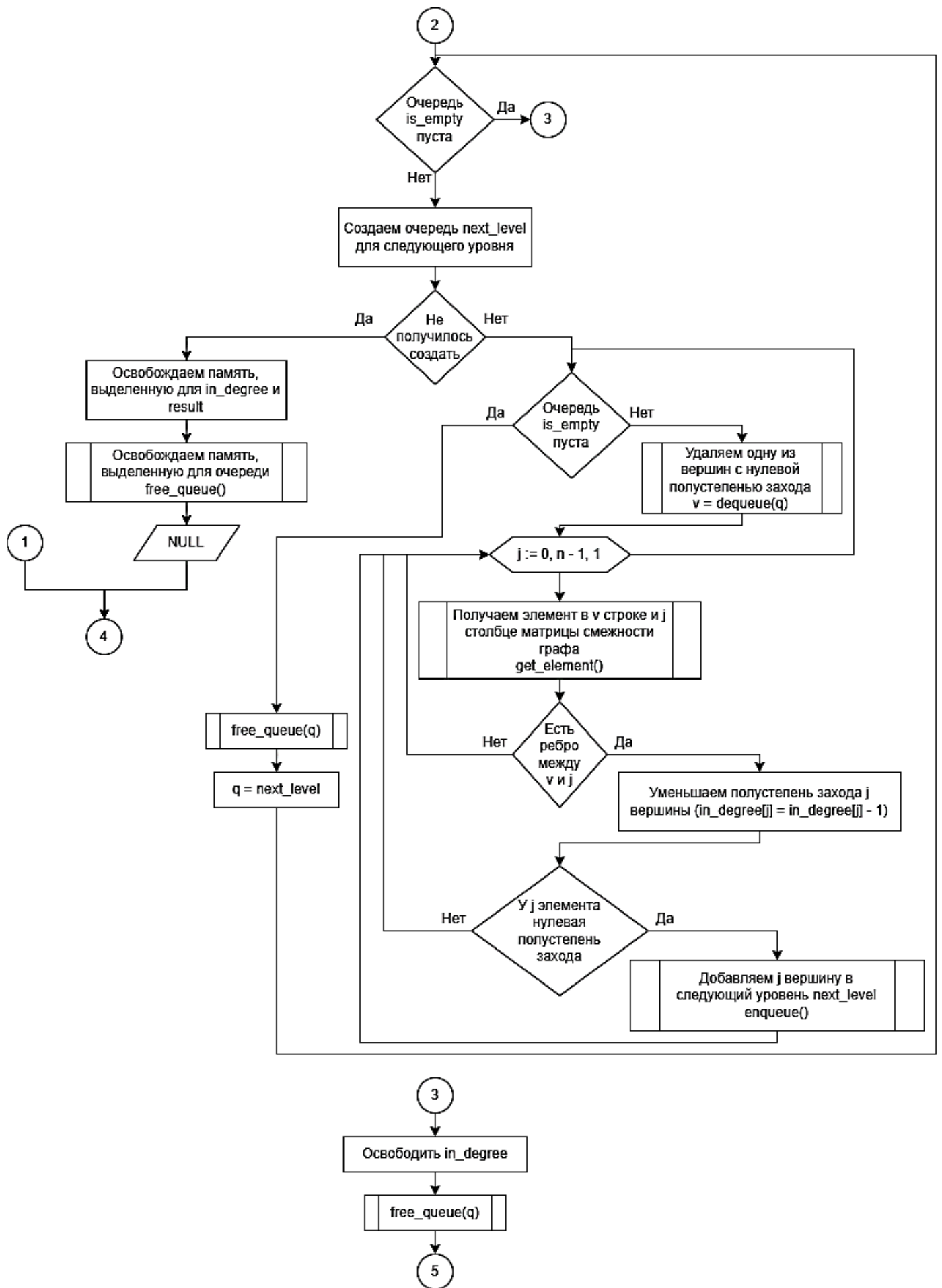
Функция `tarjan_topological_sort(DynamicQueue* matrix, int n, int* result_size)`

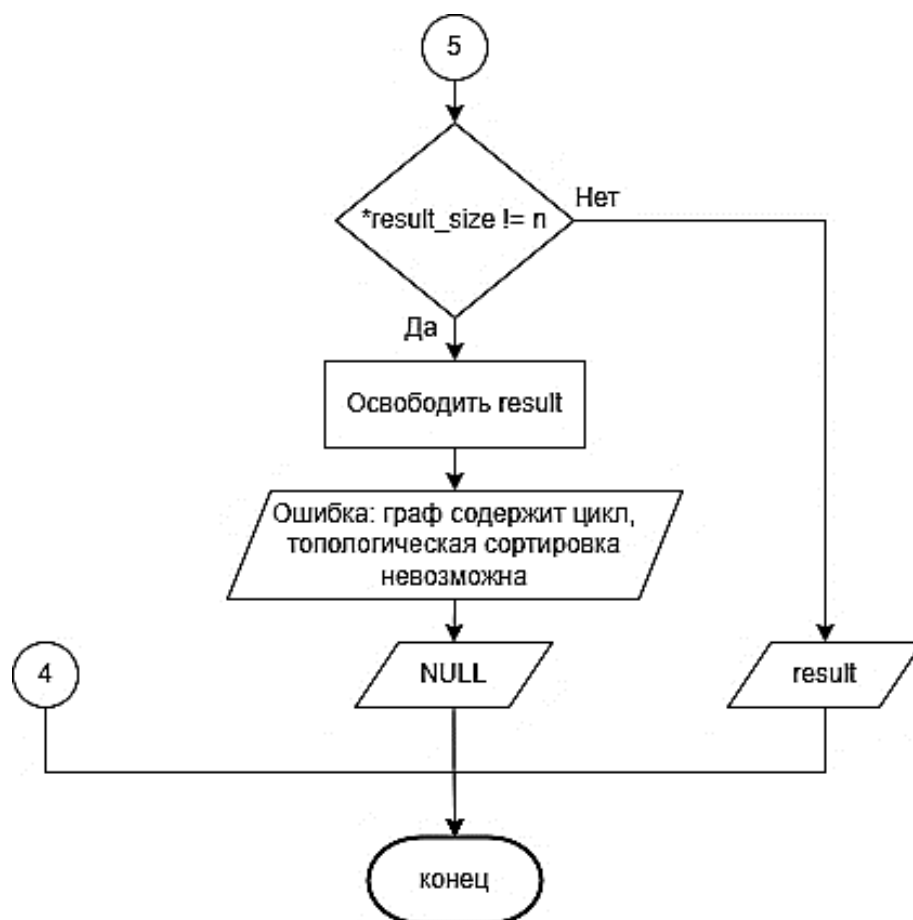




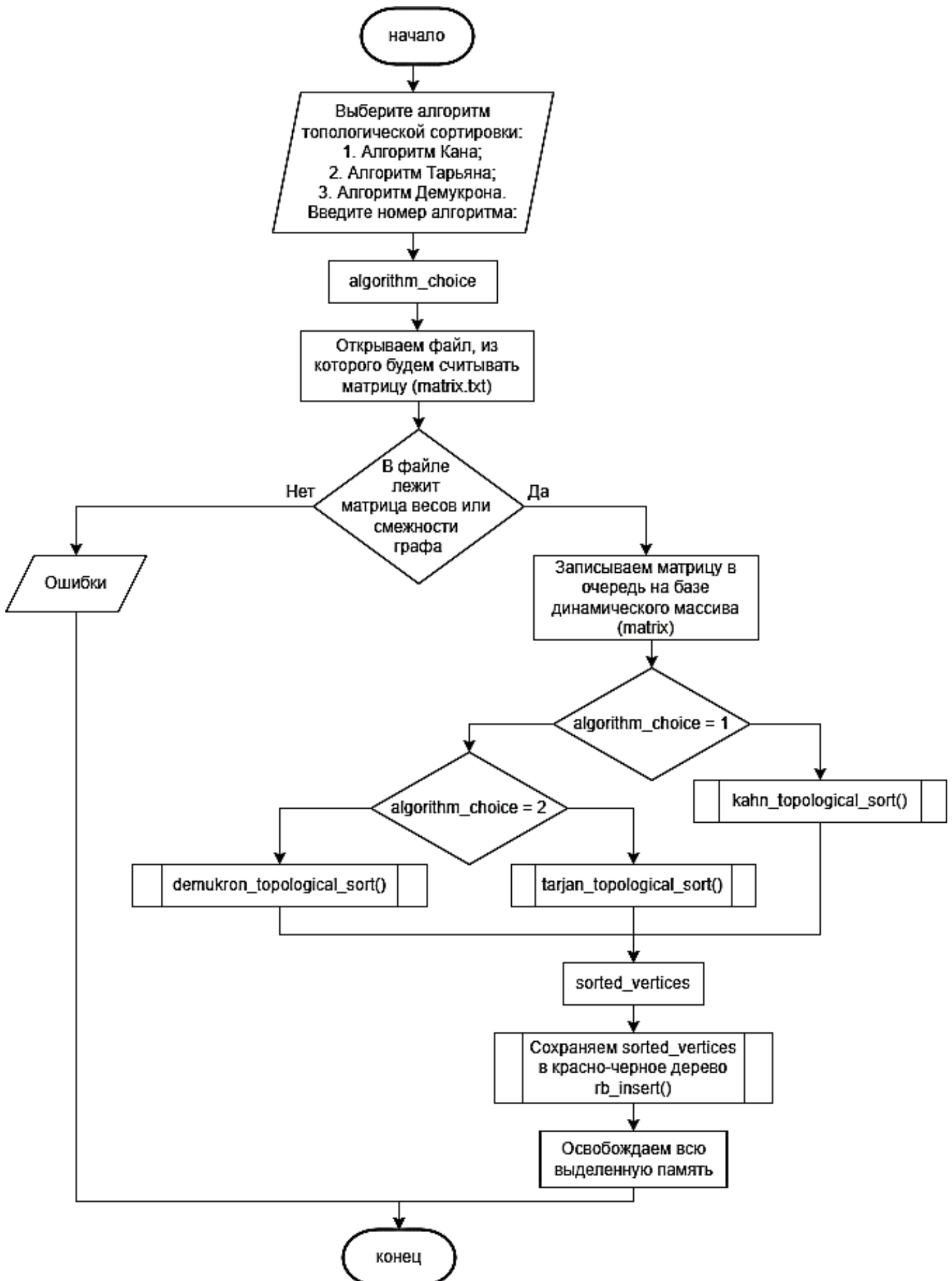
Функция `demukron_topological_sort(DynamicQueue* matrix, int n, int* result_size)`







Функция **main()**



Код на языке программирования C

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>

// Начальный размер буфера для чтения строк
#define INITIAL_BUF_SIZE 100

/*
 * Структура динамической очереди на основе массива
 * Используется для матрицы смежности и в алгоритмах сортировки
 */
typedef struct {
    int* data;    // Динамический массив для хранения элементов
    int capacity; // Текущая вместимость очереди
    int size;     // Текущее количество элементов в очереди
    int first;    // Индекс первого элемента в очереди
    int last;     // Индекс последнего элемента в очереди
} DynamicQueue;

/*
 * Перечисление для цветов узлов красно-черного дерева
 */
typedef enum { RED, BLACK } Color;

/*
 * Структура узла красно-черного дерева
 */
typedef struct RBNode {
    int value;        // Значение, хранящееся в узле
    Color color;      // Цвет узла
    struct RBNode* left; // Указатель на левого потомка
    struct RBNode* right; // Указатель на правого потомка
    struct RBNode* parent; // Указатель на родительский узел
} RBNode;

/*
 * Структура красно-черного дерева
 */
typedef struct {
    RBNode* root; // Указатель на корень дерева
    RBNode* nil;  // Специальный листовой узел
} RBTree;
```

```

// Реализация динамической очереди на основе массива

// Создание новой очереди
DynamicQueue* create_queue(int size) {
    DynamicQueue* q = malloc(sizeof(DynamicQueue)); // Выделяем память под структуру очереди
    if (!q) return NULL; // Проверка на ошибку выделения памяти

    // Выделяем память под данные очереди
    q->data = malloc(size * sizeof(int));
    if (!q->data) {
        free(q); // Освобождаем память при ошибке
        return NULL;
    }

    // Инициализация полей очереди
    q->capacity = size; // Начальная емкость
    q->size = 0; // Начальный размер (пустая очередь)
    q->first = 0; // Индекс начала
    q->last = -1; // Индекс конца (пока нет элементов)
    return q;
}

// Добавление элемента в очередь
void enqueue(DynamicQueue* q, int value) {
    // Если очередь заполнена, увеличиваем емкость
    if (q->size == q->capacity) {
        int new_capacity = q->capacity * 2; // Удваиваем емкость
        int* new_data = realloc(q->data, new_capacity * sizeof(int)); // Перевыделяем память
        if (!new_data) return; // В случае ошибки выходим

        q->data = new_data; // Обновляем указатель на данные
        q->capacity = new_capacity; // Обновляем емкость
    }

    // Вычисляем новый индекс конца очереди (кольцевой буфер)
    q->last = (q->last + 1) % q->capacity;
    q->data[q->last] = value; // Записываем значение
    q->size++; // Увеличиваем размер
}

// Извлечение элемента из очереди
int dequeue(DynamicQueue* q) {
    if (q->size == 0) return -1; // Очередь пуста

    int value = q->data[q->first]; // Получаем значение из начала
    q->first = (q->first + 1) % q->capacity; // Обновляем индекс начала (кольцевой буфер)
    q->size--; // Уменьшаем размер
    return value; // Возвращаем значение
}

```

```

}

// Проверка на пустоту очереди
bool is_empty(DynamicQueue* q) {
    return q->size == 0;
}

// Освобождение памяти очереди
void free_queue(DynamicQueue* q) {
    free(q->data); // Освобождаем массив данных
    free(q);       // Освобождаем саму структуру
}

// Работа с матрицей смежности через очередь

/*
 * Получение элемента матрицы по индексам
 * matrix - очередь, содержащая матрицу
 * row - строка
 * col - столбец
 * n - размер матрицы (n x n)
 */
int get_matrix_element(DynamicQueue* matrix, int row, int col, int n) {
    if (row < 0 || row >= n || col < 0 || col >= n) return 0; // Проверка границ

    int pos = row * n + col; // Линеаризация индексов
    int index = (matrix->first + pos) % matrix->capacity; // Учет кольцевого буфера
    return matrix->data[index]; // Возвращаем значение
}

// Реализация красно-черного дерева

// Создание нового дерева
RBTree* create_rb_tree() {
    RBTree* tree = malloc(sizeof(RBTree)); // Выделяем память под структуру
    if (!tree) return NULL; // Проверка на ошибку

    // Создаем специальный nil-узел
    tree->nil = malloc(sizeof(RBNode));
    if (!tree->nil) {
        free(tree); // Освобождаем память при ошибке
        return NULL;
    }

    // Инициализация nil-узла
    tree->nil->color = BLACK; // Nil всегда черный
    tree->nil->left = tree->nil->right = tree->nil->parent = tree->nil; // Все указатели на себя
    tree->root = tree->nil; // Корень изначально nil

```



```

return tree;
}

// Левый поворот узла x
void left_rotate(RBTree* tree, RBNode* x) {
    RBNode* y = x->right; // y - правый потомок x
    x->right = y->left; // Левый потомок y становится правым потомком x

    if (y->left != tree->nil) {
        y->left->parent = x; // Обновляем родителя левого потомка y
    }

    y->parent = x->parent; // Переносим родителя

    if (x->parent == tree->nil) {
        tree->root = y; // Если x был корнем, теперь корень - y
    } else if (x == x->parent->left) {
        x->parent->left = y; // Обновляем левого потомка родителя x
    } else {
        x->parent->right = y; // Обновляем правого потомка родителя x
    }

    y->left = x; // x становится левым потомком y
    x->parent = y; // y становится родителем x
}

// Правый поворот узла y (аналогично левому)
void right_rotate(RBTree* tree, RBNode* y) {
    RBNode* x = y->left; // x - левый потомок y
    y->left = x->right; // Правый потомок x становится левым потомком y

    if (x->right != tree->nil) {
        x->right->parent = y; // Обновляем родителя правого потомка x
    }

    x->parent = y->parent; // Переносим родителя

    if (y->parent == tree->nil) {
        tree->root = x; // Если y был корнем, теперь корень - x
    } else if (y == y->parent->right) {
        y->parent->right = x; // Обновляем правого потомка родителя y
    } else {
        y->parent->left = x; // Обновляем левого потомка родителя y
    }

    x->right = y; // y становится правым потомком x
    y->parent = x; // x становится родителем y
}

```

```

// Вставка нового значения в дерево
void rb_insert(RBTree* tree, int value) {
    // Создаем новый узел
    RBNode* z = malloc(sizeof(RBNode));
    if (!z) return; // Проверка на ошибку

    // Инициализация нового узла
    z->value = value;
    z->left = z->right = z->parent = tree->nil; // Все указатели на nil
    z->color = RED; // Новый узел всегда красный

    // Поиск места для вставки
    RBNode* y = tree->nil;
    RBNode* x = tree->root;

    while (x != tree->nil) {
        y = x;
        if (z->value < x->value) {
            x = x->left; // Идем влево
        } else {
            x = x->right; // Идем вправо
        }
    }

    z->parent = y; // Устанавливаем родителя
    if (y == tree->nil) {
        tree->root = z; // Дерево было пустое
    } else if (z->value < y->value) {
        y->left = z; // Вставляем как левого потомка
    } else {
        y->right = z; // Вставляем как правого потомка
    }

    // Балансировка дерева после вставки
    while (z != tree->root && z->parent->color == RED) {
        if (z->parent == z->parent->parent->left) {
            RBNode* y = z->parent->parent->right; // Дядя
            if (y->color == RED) {
                // Случай 1: дядя красный
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    // Случай 2: дядя черный, z - правый потомок
                    z = z->parent;
                }
            }
        }
    }
}

```

```

        left_rotate(tree, z);
    }
    // Случай 3: дядя черный, z - левый потомок
    z->parent->color = BLACK;
    z->parent->parent->color = RED;
    right_rotate(tree, z->parent->parent);
}
} else {
    // Симметричный случай
    RBNode* y = z->parent->parent->left;
    if (y->color == RED) {
        z->parent->color = BLACK;
        y->color = BLACK;
        z->parent->parent->color = RED;
        z = z->parent->parent;
    } else {
        if (z == z->parent->left) {
            z = z->parent;
            right_rotate(tree, z);
        }
        z->parent->color = BLACK;
        z->parent->parent->color = RED;
        left_rotate(tree, z->parent->parent);
    }
}
}

tree->root->color = BLACK; // Корень всегда черный
}

// Обход дерева в порядке in-order (левый-корень-правый)
void inorder_traversal(RBNode* node, RBNode* nil) {
    if (node != nil) {
        inorder_traversal(node->left, nil); // Рекурсивно обходим левое поддерево
        printf("%d ", node->value);      // Выводим значение
        inorder_traversal(node->right, nil); // Рекурсивно обходим правое поддерево
    }
}

// Вспомогательная функция для освобождения памяти дерева
void free_rb_tree_helper(RBNode* node, RBNode* nil) {
    if (node != nil) {
        free_rb_tree_helper(node->left, nil); // Рекурсивно освобождаем левое поддерево
        free_rb_tree_helper(node->right, nil); // Рекурсивно освобождаем правое поддерево
        free(node); // Освобождаем текущий узел
    }
}
}

```

```

// Освобождение памяти дерева
void free_rb_tree(RBTree* tree) {
    free_rb_tree_helper(tree->root, tree->nil); // Освобождаем все узлы
    free(tree->nil); // Освобождаем nil-узел
    free(tree);    // Освобождаем саму структуру
}

// Вспомогательные функции для работы с файлами и строками

// Проверка, является ли строка корректным числом
int is_valid_number(const char *str) {
    char *endptr;
    errno = 0;
    strtod(str, &endptr); // Пытаемся преобразовать строку в число
    // Проверяем, что преобразование завершилось успешно и нет переполнения
    return (*endptr == '\0' || isspace(*endptr)) && errno != ERANGE;
}

// Чтение строки из файла с динамическим выделением памяти
char* read_dynamic_line(FILE* file) {
    size_t buf_size = INITIAL_BUF_SIZE; // Начальный размер буфера
    char* buffer = malloc(buf_size);    // Выделяем память
    if (!buffer) return NULL;

    size_t pos = 0; // Текущая позиция в буфере
    int c;          // Считанный символ

    // Читаем символы до конца строки или файла
    while ((c = fgetc(file)) != EOF && c != '\n') {
        // Если буфер заполнен, увеличиваем его
        if (pos + 1 >= buf_size) {
            buf_size *= 2; // Удваиваем размер
            char* new_buf = realloc(buffer, buf_size); // Перевыделяем память
            if (!new_buf) {
                free(buffer);
                return NULL;
            }
            buffer = new_buf;
        }
        buffer[pos++] = (char)c; // Записываем символ
    }

    // Если ничего не прочитали и конец файла
    if (pos == 0 && c == EOF) {
        free(buffer);
        return NULL;
    }
}

```

```

    buffer[pos] = '\0'; // Завершаем строку нуль-терминатором
    return buffer;
}

// Алгоритмы топологической сортировки (адаптированные для очереди)

// Алгоритм Кана для топологической сортировки
int* kahn_topological_sort(DynamicQueue* matrix, int n, int* result_size) {
    int* in_degree = calloc(n, sizeof(int)); // Массив полустепеней захода
    if (!in_degree) return NULL;

    DynamicQueue* q = create_queue(n); // Очередь для вершин с нулевой полустепенью
    if (!q) {
        free(in_degree);
        return NULL;
    }

    int* result = malloc(n * sizeof(int)); // Результат сортировки
    if (!result) {
        free(in_degree);
        free_queue(q);
        return NULL;
    }

    *result_size = 0; // Изначально размер результата 0

    // Вычисляем полустепени захода для всех вершин
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (get_matrix_element(matrix, i, j, n)) {
                in_degree[j]++; // Увеличиваем для вершины j, если есть ребро i->j
            }
        }
    }

    // Добавляем вершины с нулевой полустепенью в очередь
    for (int i = 0; i < n; i++) {
        if (in_degree[i] == 0) {
            enqueue(q, i);
        }
    }

    // Основной цикл алгоритма
    while (!is_empty(q)) {
        int v = dequeue(q); // Извлекаем вершину
        result[*result_size++] = v; // Добавляем в результат

        // Уменьшаем полустепени захода для всех соседей

```

```

    for (int j = 0; j < n; j++) {
        if (get_matrix_element(matrix, v, j, n)) {
            in_degree[j]--;
            if (in_degree[j] == 0) {
                enqueue(q, j); // Если полустепень стала нулевой, добавляем в очередь
            }
        }
    }
}

// Освобождаем память
free(in_degree);
free_queue(q);

// Проверяем, был ли граф ациклическим
if (*result_size != n) {
    free(result);
    printf("Ошибка: граф содержит цикл, топологическая сортировка невозможна\n");
    return NULL;
}

return result;
}

// Вспомогательная функция для алгоритма Тарьяна (рекурсивный DFS)
int tarjan_dfs(int v, DynamicQueue* matrix, int n, int* colors, DynamicQueue* result) {
    colors[v] = 1; // Помечаем вершину как посещенную (серую)

    // Обходим всех соседей
    for (int u = 0; u < n; u++) {
        if (get_matrix_element(matrix, v, u, n)) {
            if (colors[u] == 1) {
                // Найден цикл (серая вершина в стеке вызовов)
                return 0;
            }
            if (colors[u] == 0) {
                if (!tarjan_dfs(u, matrix, n, colors, result)) {
                    return 0; // Распространяем ошибку цикла
                }
            }
        }
    }

    enqueue(result, v); // Добавляем вершину в результат (после обработки всех потомков)
    colors[v] = 2; // Помечаем как обработанную (черную)
    return 1; // Успешное завершение
}

```

```

// Алгоритм Тарьяна для топологической сортировки с проверкой на циклы
int* tarjan_topological_sort(DynamicQueue* matrix, int n, int* result_size) {
    int* colors = calloc(n, sizeof(int)); // Массив цветов вершин (0 - белый, 1 - серый, 2 - черный)
    if (!colors) return NULL;

    DynamicQueue* q = create_queue(n); // Очередь для хранения результата
    if (!q) {
        free(colors);
        return NULL;
    }

    int has_cycle = 0; // Флаг наличия цикла

    // Запускаем DFS для всех белых вершин
    for (int i = 0; i < n; i++) {
        if (colors[i] == 0) {
            if (!tarjan_dfs(i, matrix, n, colors, q)) {
                has_cycle = 1;
                break;
            }
        }
    }

    free(colors); // Освобождаем массив цветов

    // Если обнаружен цикл
    if (has_cycle) {
        printf("Ошибка: граф содержит цикл, топологическая сортировка невозможна\n");
        free_queue(q);
        return NULL;
    }

    *result_size = q->size; // Размер результата равен размеру очереди
    int* result = malloc(*result_size * sizeof(int)); // Выделяем память под результат
    if (!result) {
        free_queue(q);
        return NULL;
    }

    // Переносим элементы из очереди в массив результата в обратном порядке
    for (int i = *result_size - 1; i >= 0; i--) {
        result[i] = dequeue(q);
    }

    free_queue(q); // Освобождаем очередь
    return result;
}

```

```

// Алгоритм Демукрона для топологической сортировки
int* demukron_topological_sort(DynamicQueue* matrix, int n, int* result_size) {
    int* in_degree = calloc(n, sizeof(int)); // Массив полустепеней захода
    if (!in_degree) return NULL;

    DynamicQueue* q = create_queue(n); // Очередь для текущего уровня
    if (!q) {
        free(in_degree);
        return NULL;
    }

    int* result = malloc(n * sizeof(int)); // Результат сортировки
    if (!result) {
        free(in_degree);
        free_queue(q);
        return NULL;
    }

    *result_size = 0; // Изначально размер результата 0

    // Вычисляем полустепени захода для всех вершин
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (get_matrix_element(matrix, i, j, n)) {
                in_degree[j]++;
            }
        }
    }

    // Основной цикл алгоритма
    while (true) {
        // Добавляем все вершины с нулевой полустепенью на текущем уровне
        for (int i = 0; i < n; i++) {
            if (in_degree[i] == 0) {
                enqueue(q, i);
                in_degree[i] = -1; // Помечаем, чтобы не добавлять снова
            }
        }

        if (is_empty(q)) break; // Если очередь пуста, завершаем

        // Обрабатываем текущий уровень
        while (!is_empty(q)) {
            int v = dequeue(q); // Извлекаем вершину
            result[(*result_size)++] = v; // Добавляем в результат

            // Уменьшаем полустепени захода для всех соседей
            for (int j = 0; j < n; j++) {

```



```

        if (get_matrix_element(matrix, v, j, n)) {
            in_degree[j]--;
        }
    }
}

// Освобождаем память
free(in_degree);
free_queue(q);

// Проверяем, был ли граф ациклическим
if (*result_size != n) {
    free(result);
    printf("Ошибка: граф содержит цикл, топологическая сортировка невозможна\n");
    return NULL;
}

return result;
}

// Основная функция программы
int main() {
    // Выбор алгоритма сортировки
    printf("Выберите алгоритм топологической сортировки:\n");
    printf("1. Алгоритм Кана\n");
    printf("2. Алгоритм Тарьяна\n");
    printf("3. Алгоритм Демукрона\n");
    printf("Введите номер алгоритма: ");

    int algorithm_choice;
    if (scanf("%d", &algorithm_choice) != 1 || algorithm_choice < 1 || algorithm_choice > 3) {
        printf("Ошибка: некорректный выбор алгоритма\n");
        return 1;
    }

    // Открываем файл с матрицей смежности
    const char* filename = "matrix.txt";
    FILE* file = fopen(filename, "r");
    if (!file) {
        perror("Ошибка: не удалось открыть файл");
        return 1;
    }

    // Первый проход: определяем размеры матрицы
    int rows = 0, cols = 0;
    char* line = NULL;

```

```

while ((line = read_dynamic_line(file)) != NULL) {
    if (strlen(line) == 0) { // Пропускаем пустые строки
        free(line);
        continue;
    }

    int current_cols = 0; // Счетчик столбцов в текущей строке
    char* ptr = line;
    char* token;

    // Разбиваем строку на токены (числа)
    while (*ptr) {
        while (*ptr && isspace(*ptr)) ptr++; // Пропускаем пробелы
        if (!*ptr) break; // Конец строки

        token = ptr;
        while (*ptr && !isspace(*ptr)) ptr++; // Ищем конец числа

        // Проверяем, является ли токен корректным числом
        if (!is_valid_number(token)) {
            printf("Ошибка: некорректное значение '%s'\n", (int)(ptr - token), token);
            free(line);
            fclose(file);
            return 1;
        }
        current_cols++;
    }

    // Проверяем согласованность количества столбцов
    if (cols == 0) {
        cols = current_cols; // Первая строка определяет количество столбцов
    } else if (current_cols != cols) {
        printf("Ошибка: разное количество элементов в строках\n");
        free(line);
        fclose(file);
        return 1;
    }
    rows++; // Увеличиваем счетчик строк
    free(line); // Освобождаем строку
}

// Проверяем, что матрица не пустая и квадратная
if (rows == 0 || cols == 0) {
    printf("Ошибка: пустая матрица\n");
    fclose(file);
    return 1;
}

```

```

if (rows != cols) {
    printf("Ошибка: матрица смежности должна быть квадратной\n");
    fclose(file);
    return 1;
}

// Второй проход: заполняем матрицу смежности
rewind(file); // Возвращаемся в начало файла
DynamicQueue* matrix = create_queue(rows); // Создаем очередь для матрицы
if (!matrix) {
    fclose(file);
    return 1;
}

// Читаем файл снова и заполняем матрицу
while ((line = read_dynamic_line(file)) != NULL) {
    if (strlen(line) == 0) {
        free(line);
        continue;
    }

    char* ptr = line;
    char* token;

    // Разбираем строку на числа
    while (*ptr) {
        while (*ptr && isspace(*ptr)) ptr++;
        if (!*ptr) break;

        token = ptr;
        while (*ptr && !isspace(*ptr)) ptr++;

        // Преобразуем строку в число и добавляем в матрицу
        double value = strtod(token, NULL);
        enqueue(matrix, (value != 0) ? 1 : 0); // Ненулевые значения становятся 1
    }
    free(line); // Освобождаем строку
}
fclose(file); // Закрываем файл

// Выводим матрицу смежности для проверки
printf("Матрица смежности графа (%dx%d):\n", rows, rows);
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < rows; j++) {
        printf("%d ", get_matrix_element(matrix, i, j, rows));
    }
    printf("\n");
}

```

```

// Выполняем топологическую сортировку выбранным алгоритмом
int result_size;
int* sorted_vertices = NULL;

switch (algorithm_choice) {
    case 1:
        printf("Используется алгоритм Кана\n");
        sorted_vertices = kahn_topological_sort(matrix, rows, &result_size);
        break;
    case 2:
        printf("Используется алгоритм Тарьяна\n");
        sorted_vertices = tarjan_topological_sort(matrix, rows, &result_size);
        break;
    case 3:
        printf("Используется алгоритм Демукрона\n");
        sorted_vertices = demukron_topological_sort(matrix, rows, &result_size);
        break;
}

// Проверяем успешность сортировки
if (!sorted_vertices) {
    free_queue(matrix);
    return 1;
}

// Выводим результат сортировки
printf("Отсортированные вершины: ");
for (int i = 0; i < result_size; i++) {
    printf("%d ", sorted_vertices[i] + 1); // +1 для удобочитаемости (вершины нумеруются с 1)
}
printf("\n");

// Сохраняем результат в красно-черное дерево
RBTree* tree = create_rb_tree();
if (!tree) {
    printf("Ошибка: не удалось создать красно-черное дерево\n");
    free(sorted_vertices);
    free_queue(matrix);
    return 1;
}

// Вставляем все вершины в дерево
for (int i = 0; i < result_size; i++) {
    rb_insert(tree, sorted_vertices[i] + 1);
}

//Вывод дерева

```

```
printf("Красно-черное дерево (вершины в порядке возрастания): ");
inorder_traversal(tree->root, tree->nil);
printf("\n");

// Освобождаем память
free(sorted_vertices);
free_queue(matrix);
free_rb_tree(tree);

return 0;
}
```

Примеры работы программы

C: > Users > Анастасия > matrix.txt

```
1
2 | 0 102 1 -0.123 0.1
3
4 0 0 0 -5 0
5 0 0 0 12568979 1
6 0 0 0 1.025956565
7 0 0 0 0 0
8
9
10
11
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Анастасия> gcc lab_4_algorithms.c -o program
PS C:\Users\Анастасия> ./program
Выберите алгоритм топологической сортировки:
1. Алгоритм Кана
2. Алгоритм Тарьяна
3. Алгоритм Демукрона
Введите номер алгоритма: 1
Матрица смежности графа (5x5):
0 1 1 1 1
0 0 0 1 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0
Используется алгоритм Кана
Отсортированные вершины: 1 2 3 4 5
Красно-черное дерево (вершины в порядке возрастания): 1 2 3 4 5
PS C:\Users\Анастасия> ./program
Выберите алгоритм топологической сортировки:
1. Алгоритм Кана
2. Алгоритм Тарьяна
3. Алгоритм Демукрона
Введите номер алгоритма: 2
Матрица смежности графа (5x5):
0 1 1 1 1
0 0 0 1 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0
Используется алгоритм Тарьяна
Отсортированные вершины: 1 3 2 4 5
Красно-черное дерево (вершины в порядке возрастания): 1 2 3 4 5
PS C:\Users\Анастасия> ./program
Выберите алгоритм топологической сортировки:
1. Алгоритм Кана
2. Алгоритм Тарьяна
3. Алгоритм Демукрона
Введите номер алгоритма: 3
Матрица смежности графа (5x5):
0 1 1 1 1
0 0 0 1 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0
Используется алгоритм Демукрона
Отсортированные вершины: 1 2 3 4 5
Красно-черное дерево (вершины в порядке возрастания): 1 2 3 4 5
```